# CO1-AT1-Part 2

## Team 5 — Airline Reservation System

**Name** T G Jai Kousick

**Registration No.** 192421367

**Subject** DevSecOps Engineering and its Application Security **Subject Code**

CSA5802

## Background

SkyFly Airlines continuously updates its online booking platform with promotional offers and new services. The application depends heavily on open-source libraries, but no formal process exists for checking known vulnerabilities. Patches are delayed to avoid interrupting business operations, API protection is inconsistent, and security reviews are reactive rather than proactive. A recent outage disrupted online ticket bookings, causing revenue loss and customer complaints. This report investigates the underlying security weaknesses and proposes a DevSecOps-based remediation strategy.

## 1. Security Risks Identified

• **Unmanaged open-source dependencies:** No Software Composition Analysis (SCA) process exists to track known CVEs in third-party libraries used by the booking platform.

• **Delayed patch management:** Security patches are postponed to avoid business disruption, leaving known vulnerabilities exploitable for extended periods. • **Inconsistent API security:** APIs that expose booking, payment, and customer data lack uniform

authentication, authorization, and rate-limiting controls.

• **Reactive security posture:** Security reviews happen only after incidents occur instead of being embedded throughout the development lifecycle.

• **Lack of continuous vulnerability scanning in CI/CD:** No automated security gates exist in the build/release pipeline, allowing insecure code to reach production. • **Weak monitoring and incident response readiness:** The recent outage shows limited visibility into system health and attack indicators, delaying detection and recovery.

## 2. Vulnerabilities Identified and Classification

| | | | |
|---|---|---|---|
| 1 | Outdated / vulnerable open-source components | Libraries with publicly known CVEs are used without patching or version tracking. | High |
| 2 | Insecure / exposed API endpoints | Booking and payment APIs lack consistent authentication, input validation, and rate limiting. | High |
| 3 | Sensitive data exposure | Customer PII and payment data may be transmitted or stored without adequate encryption controls. | High |
| 4 | Injection flaws (SQL/NoSQL/Command) | Insufficient input sanitization in booking forms and search fields can allow injection attacks. | High |
| 5 | Delayed patch/update cycle | Known vulnerabilities remain unpatched for long periods due to business-continuity concerns. | Medium |

| | | | |
|---|---|---|---|
| 6 | Misconfigured cloud/server settings | Default configurations, open ports, or excessive permissions on hosting infrastructure. | Medium |
| 7 | Absence of automated security testing (SAST/DAST/SCA) in CI/CD | No scanning gates before deployment, allowing vulnerable code to reach production. | Medium |
| 8 | Insufficient logging and monitoring | Limited audit trails delay detection of anomalies and slow incident response. | Low |

## 3. Possible Cyber Threats and Their Impact

| | | |
|---|---|---|
| Distributed Denial of Service (DDoS) | Attackers flood booking servers/APIs with traffic. | Service outage, exactly like the recent incident — lost bookings, revenue loss, reputational damage. |
| SQL/NoSQL Injection | Exploiting unsanitized input in search or booking forms. | Unauthorized access to customer/payment databases, data theft or corruption. |
| Supply-chain attack via vulnerable open-source library | A known CVE in an unpatched dependency is exploited. | Full application or server compromise, remote code execution. |
| Credential stuffing / brute force | Automated login attempts using leaked credential lists against weak API auth. | Account takeover, fraudulent bookings, loss of customer trust. |
| Man-in-the-Middle (MITM) | Interception of API traffic lacking strong TLS/encryption enforcement. | Theft of payment card data and session tokens. |
| Ransomware | Malware introduced via a compromised dependency or unpatched server. | System lockout, extended downtime, extortion demands. |
| API abuse / broken object level authorization | Inconsistent API protection allows access to another user's booking records. | Privacy violations, regulatory penalties, customer data leakage. |

## 4. DevOps vs DevSecOps — Comparison Table

| | | |
|---|---|---|
| Primary Focus | Speed of delivery and collaboration between Dev and Ops | Speed of delivery with security built into every stage |
| Security Ownership | Handled by a separate security team, usually near release | Shared responsibility across Dev, Ops, and Security teams |
| When Security Is Applied | Late in the lifecycle (post-development / pre-release) | Continuously, from design and coding through deployment ('shift-left') |

| | | |
|---|---|---|
| Testing Approach | Functional and performance testing emphasized | SAST, DAST, SCA, and container scanning integrated into pipeline |
| Tools Used | CI/CD tools (Jenkins, GitLab CI), monitoring tools | CI/CD tools plus security tools (SonarQube, OWASP ZAP, Snyk, Trivy) |
| Culture | Collaboration between developers and operations | Security-first culture involving developers, ops, and security engineers |
| Vulnerability Management | Reactive — patched after discovery or incident | Proactive — automated scanning flags issues before release |
| Compliance & Governance | Addressed separately, often manually | Built into automated pipelines with policy-as-code checks |
| Incident Response | Reactive, after outage or breach is reported | Continuous monitoring enables early detection and faster response |
| Business Outcome | Faster releases but higher residual security risk | Faster and safer releases, reduced breach/downtime risk |

## 5. Recommended OWASP Security Best Practices

• **Adopt the OWASP Top 10 as a secure coding baseline:** Train developers to prevent injection, broken authentication, and security misconfiguration issues at the code level. • **Use Software Composition Analysis (SCA) tools (e.g., OWASP Dependency-Check):** Continuously scan open-source libraries for known CVEs and enforce timely patching policies.

• **Follow OWASP ASVS (Application Security Verification Standard):** Strengthen authentication, session management, and access control across the booking platform. • **Apply the OWASP API Security Top 10:** Enforce consistent authentication, authorization, input validation, and rate limiting across all booking and payment APIs.

• **Implement input validation and output encoding (OWASP guidance):** Prevent SQL injection and cross-site scripting in booking forms and search fields, combined with routine OWASP ZAP/Burp DAST scans.

## 6. Business Impact if Issues Remain Unresolved

If the identified risks and vulnerabilities are not addressed, SkyFly Airlines faces compounding business consequences:

• **Revenue loss:** Repeated outages (like the recent one) directly block ticket sales and drive customers to competitor platforms.

• **Reputational damage:** Data breaches or downtime erode customer trust and brand credibility in a highly competitive travel market.

• **Regulatory and legal exposure:** Exposure of payment or personal data can trigger penalties under data protection and PCI-DSS compliance requirements. • **Increased operational cost:** Reactive firefighting after incidents is more expensive than proactive prevention, including breach investigation and remediation costs. • **Loss of competitive advantage:** Delayed, insecure releases slow innovation while competitors ship new features safely and faster.

• **Customer churn and loss of loyalty:** Frequent service disruption pushes frequent flyers toward more reliable booking platforms.

## 7. Concept Map: Risk → Vulnerability → DevSecOps Practice → Business Benefit

The chain below illustrates how each identified risk traces through a specific vulnerability, is mitigated by a DevSecOps practice, and ultimately produces a measurable business benefit.

| | | | |
|---|---|---|---|
| Unmanaged open-source dependencies | Outdated components with known CVEs | Automated SCA scanning (OWASP Dependency-Check / Snyk) in CI/CD | Reduced breach risk; fewer emergency patches |
| Delayed patch management | Long-lived unpatched vulnerabilities | Continuous patch automation with policy-as-code gating releases | Faster, safer releases; lower downtime risk |
| Inconsistent API security | Broken authentication / missing rate limits | OWASP API Security Top 10 controls + API gateway policies | Protected customer data; fewer account takeovers |
| Reactive security reviews | Injection flaws found late or in production | Shift-left SAST/DAST integrated into every sprint | Early defect detection; lower remediation cost |
| No CI/CD security gates | Vulnerable code reaches production | Security gates (build fails on critical findings) | Higher release confidence; regulatory compliance |

| | | | |
|---|---|---|---|
| Weak monitoring | Slow incident detection (as in the outage) | Continuous monitoring, logging, and alerting (DevSecOps observability) | Faster recovery; minimized revenue/reputation loss |

**Flow summary:** Each unresolved *Risk* creates a specific technical *Vulnerability*. Embedding a targeted *DevSecOps Practice* into the pipeline closes that vulnerability, which in turn delivers a direct, measurable *Business Benefit* — closing the loop from technical weakness to business value.

## 8. Five-Minute Presentation Summary

The following outline summarizes the investigation for a five-minute team presentation (approx. 30–40 seconds per slide):

**Slide 1 — Introduction (30s)**
SkyFly Airlines' booking platform outage and its root cause: absent vulnerability management and reactive security.

**Slide 2 — Security Risks (45s)**
Unmanaged dependencies, delayed patching, inconsistent API protection, reactive reviews, weak monitoring.

**Slide 3 — Key Vulnerabilities (45s)**
High-severity: vulnerable open-source components, insecure APIs, sensitive data exposure, injection flaws.

**Slide 4 — Cyber Threats & Impact (45s)**
DDoS causing outage, injection attacks, supply-chain exploits, credential stuffing, and their business consequences.

**Slide 5 — DevOps vs DevSecOps (45s)**
Shift from reactive, late-stage security to continuous, shift-left security integrated into the pipeline.

**Slide 6 — OWASP Best Practices (45s)**
Top 10 adoption, SCA scanning, ASVS, API Security Top 10, input validation/output encoding.

**Slide 7 — Business Impact (30s)**
Revenue loss, reputational damage, regulatory exposure, customer churn if left unresolved.

**Slide 8 — Concept Map & Recommendation (30s)**
Risk → Vulnerability → DevSecOps Practice → Business Benefit — adopt DevSecOps to convert security fixes into business value.

**Slide 9 — Conclusion (10s)**

Recommend immediate adoption of a DevSecOps pipeline with automated scanning, patch  policy, and continuous monitoring.